

# 1.原理与补充知识

## 1.big.Int 的一些常见方法和属性:

- `SetInt64(x int64)`: 将一个 `int64` 类型的整数赋值给 `big.Int`。
- `SetString(s string, base int)`: 将一个字符串表示的整数按照指定的进制转换为 `big.Int`。
- `SetBytes(x []byte)(s string)`: 将一个字节型变量转换成 `big.Int`
- `Add(x, y *big.Int) *big.Int`: 将两个 `big.Int` 相加, 并返回结果。
- `Sub(x, y *big.Int) *big.Int`: 将一个 `big.Int` 减去另一个 `big.Int`, 并返回结果。
- `Mul(x, y *big.Int) *big.Int`: 将两个 `big.Int` 相乘, 并返回结果。
- `Div(x, y, z *big.Int) *big.Int`: 将一个 `big.Int` 除以另一个 `big.Int`, 并返回商。
- `Mod(x, y, z *big.Int) *big.Int`: 将一个 `big.Int` 对另一个 `big.Int` 取模, 并返回结果。
- `Cmp(y *big.Int) int`: 将 `big.Int` 与另一个 `big.Int` 比较, 返回 -1、0 或 1, 分别表示小于、等于或大于。

## 2.Target, nonce和PoW

target本质上是一个和SHA256生成哈希值一样大的整数

特殊的是这个数前几位都是0

我们要求, 通过反复枚举nonce, 生成一个小于target的数字。这样新产生的区块是成立的 (即前面0个数大于等于)

这一过程被称为: 工作量证明 (PoW算法)

## 3.SHA256的充足性

sha256生成的哈希值256二进制位, 有大约是 $1.157920892 \times 10^{77}$ 种可能

而我们要求一个区块链中的区块哈希值不能相同, 这个算法产生的值重复的概率几乎为零

哪怕是满足target限制的hash值也是

## 2.源代码

```
package main

import (
    "bytes"
    "crypto/sha256"
    "encoding/binary"
    "fmt"
    "log"
    "math"
    "math/big"
    "time"
)
```

```

type Block struct {
    Timestamp int64
    Hash      []byte
    PrevHash  []byte
    Target    []byte
    Nonce     int64
    Data      []byte
}

const (
    Difficulty = 12
)

func (b *Block) GetTarget() []byte {
    target := big.NewInt(1)
    target.Lsh(target, uint(256-Difficulty))
    return target.Bytes()
}

func (b *Block) GetBase4Nonce(nonce int64) []byte {
    data := bytes.Join([][]byte{
        ToHexInt(b.Timestamp),
        b.PrevHash,
        ToHexInt(int64(nonce)),
        b.Target,
        b.Data,
    },
        []byte{},
    )
    return data
}

func (b *Block) FindNonce() int64 {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    var nonce int64
    nonce = 0
    intTarget.SetBytes(b.Target)

    for nonce < math.MaxInt64 {
        data := b.GetBase4Nonce(nonce)
        hash = sha256.Sum256(data)
        intHash.SetBytes(hash[:])
        if intHash.Cmp(&intTarget) == -1 {
            break
        } else {
            nonce++
        }
    }
    return nonce
}

func (b *Block) validatePow() bool {
    var intHash big.Int
    var intTarget big.Int

```

```

    var hash [32]byte
    intTarget.SetBytes(b.Target)
    data := b.GetBase4Nonce(b.Nonce)
    hash = sha256.Sum256(data)
    intHash.SetBytes(hash[:])
    if intHash.Cmp(&intTarget) == -1 {
        return true
    }
    return false
}

func Handle(err error) {
    if err != nil {
        log.Panic(err)
    }
}

func ToHexInt(num int64) []byte {
    buff := new(bytes.Buffer)
    err := binary.Write(buff, binary.BigEndian, num)
    Handle(err)
    return buff.Bytes()
}

type Blockchain struct {
    Blocks []*Block
}

func (bc *Blockchain) AddBlock(data string) {
    newBlock := CreateBlock(bc.Blocks[len(bc.Blocks)-1].Hash, []byte(data))
    bc.Blocks = append(bc.Blocks, newBlock)
}

func CreateBlockchain() *Blockchain {
    blockchain := Blockchain{}
    blockchain.Blocks = append(blockchain.Blocks, GenesisBlock())
    return &blockchain
}

func (b *Block) SetHash() {
    information := bytes.Join([][]byte{ToHexInt(b.Timestamp), b.PrevHash,
    b.Target, ToHexInt(b.Nonce), b.Data}, []byte{})
    hash := sha256.Sum256(information)
    b.Hash = hash[:]
}

func CreateBlock(prevhash, data []byte) *Block {
    block := Block{time.Now().Unix(), []byte{}, prevhash, []byte{}, 0, data}
    block.Target = block.GetTarget()
    block.Nonce = block.FindNonce()
    block.SetHash()
    return &block
}

```

```

func GenesisBlock() *Block {
    genesiswords := "Hello, blockchain!"
    return CreateBlock([]byte{}, []byte(genesiswords))
}

func main() {
    chain := CreateBlockchain()
    time.Sleep(time.Second)
    chain.AddBlock("After genesis, I have something to say.")
    time.Sleep(time.Second)
    chain.AddBlock("Leo Cao is awesome!")
    time.Sleep(time.Second)
    chain.AddBlock("I can't wait to follow his github!")
    time.Sleep(time.Second)

    for _, block := range chain.Blocks {
        fmt.Printf("Timestamp: %d\n", block.Timestamp)
        fmt.Printf("hash: %x\n", block.Hash)
        fmt.Printf("Previous hash: %x\n", block.PrevHash)
        fmt.Printf("nonce: %d\n", block.Nonce)
        fmt.Printf("data: %s\n", block.Data)
        fmt.Println("Proof of work validation:", block.ValidatePow())
    }
}

```

## 3.分块分析

### 1.导入包作用

```

import (
    "bytes"
    "crypto/sha256"
    "encoding/binary"
    "fmt"
    "log"
    "math"

    //提供了基本函数  $\pi$  e登常量 等等计算功能
    //加密货币中的加密算法、共识机制和地址生成算法都可能涉及到数学运算
    "math/big"
    //这个包提供了大数运算的支持
    //加密货币中的椭圆曲线加密算法中经常需要处理大数
    //区块链中的一些计算可能涉及到非常大的数字，比如计算工作量证明的难度目标值
    "time"
)

```

## 2.区块包含元素作用

```
type Block struct {
    Timestamp int64
    Hash      []byte
    PrevHash  []byte
    Target    []byte
    //Target 字段通常用于表示工作量证明（Proof of Work, PoW）算法中的难度目标值。
    //在Pow算法中，矿工需要不断尝试不同的Nonce值，使得区块的哈希值小于或等于Target，以此来满足网络的难度要求
    //区块头中，Target字段表示了当前区块的目标哈希值，矿工需要通过不断调整Nonce值来寻找满足条件的哈希值，从而完成区块的挖掘工作
    Nonce int64
    //Nonce 字段是一个整数值，用于在Pow算法中尝试寻找有效哈希值的过程中进行递增或变化。
    //矿工通过调整Nonce值来产生不同的区块哈希，以尝试满足难度目标。
    Data []byte
}
```

## 3.难度

```
const (
    Difficulty = 12 //设置难度
)
```

difficulty 反映了矿工找到下一个有效区块的难易程度，难度随区块头目标Hash值（target）的变动而变动，target值越小，难度越大。

## 4.生成Target

```
// 在区块结构体（Block）中获取难度目标（Target）的字节数组表示。
func (b *Block) GetTarget() []byte {
    //区块头目标Hash值（target） target值越小，难度越大
    target := big.NewInt(1)
    //NewInt 大整数对象是一种能够存储任意精度整数的数据类型，用于处理超出普通整数范围的大数字计算
    target.Lsh(target, uint(256-Difficulty))
    //Lsh 方法是左移操作，用于将 target 左移 256-Difficulty 位
    return target.Bytes()
}
```

如果我把区块链比作 设置挖矿目标和挖矿两个部分，那么这个GetTarget函数实现的是那个部分？

显然是设置难度

## 5.所有数据连成字节

```
// 将区块的时间戳（Timestamp）、前一个区块的哈希值（PrevHash）、给定的 nonce、挖矿目标值（Target）、以及区块数据（Data）连接成字节返回
func (b *Block) GetBase4Nonce(nonce int64) []byte {
    //(b *Block) 是一个方法接收器（receiver），它定义了一个方法 GetBase4Nonce() 与 Block 结构体相关联
    //指针类型接收器 意味着这个方法可以在 Block 类型的实例上被调用，并且可以修改这个实例的状态
    data := bytes.Join([][]byte{
        ToHexInt(b.Timestamp),
        b.PrevHash,
        ToHexInt(int64(nonce)),
        b.Target,
        b.Data,
    }, //字节切片被放置在一个外部的切片 [][]byte{} 中，以便稍后使用 bytes.Join 函数将它们连接起来，形成一个单独的字节切片
        []byte{},
    )
    return data
}
```

## 6.nonce枚举产生合法区块

```
func (b *Block) FindNonce() int64 {
    var intHash big.Int //哈希值
    //big.Int 是 Go 语言标准库 math/big 中的一个结构体类型，用于表示任意精度的整数
    var intTarget big.Int //要求挖矿大小target的值
    var hash [32]byte
    var nonce int64 //nonce变量，试图通过枚举来实现区块的创建Pow验证合法
    nonce = 0
    intTarget.SetBytes(b.Target)

    for nonce < math.MaxInt64 {
        // 将赋值语句放在函数体内部
        data := b.GetBase4Nonce(nonce)
        //调用函数， 传入nonce 返回一个包含所有数据的切片
        hash = sha256.Sum256(data) //使用SHA-256 计算data切片的 哈希值
        intHash.SetBytes(hash[:]) //将 hash 数组转换为 big.Int 类型，并赋值
        给 intHash, 准备与目标值进行比较
        if intHash.Cmp(&intTarget) == -1 { //big.Int 类型的方法 Cmp() 来比较两个大整数 intHash 和 intTarget 的大小
            break
        } else {
            nonce++
        }
    }
    return nonce
} //这个函数在创建block节点中用到
```

## 7.判断节点是否合法

```
// 通过hash和target的比较 判断节点是否合法
func (b *Block) validatePow() bool {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    intTarget.SetBytes(b.Target)
    //将字节数组 b.Target 的值设置给 intTarget 变量
    data := b.GetBase4Nonce(b.Nonce)
    hash = sha256.Sum256(data)
    intHash.SetBytes(hash[:])
    if intHash.Cmp(&intTarget) == -1 {
        return true
    }
    return false
}
```

## 8.处理错误

```
// 用于处理错误的通用函数 Handle(err error) 作用是在出现错误时，记录错误信息并终止程序的执行
func Handle(err error) {
    if err != nil {
        log.Panic(err)
    }
}
```

## 运行结果

```
go run main.go
```

```
Timestamp: 1711788684
hash: d21ec302d0aa322a933947b2020419e054627dbb0bf074d56deb42b862abedcb
Previous hash:
nonce: 14124
data: Hello, blockchain!
Proof of work validation: true
Timestamp: 1711788685
hash: 6f3c44e96d073df10d86ec1f2c609795cffe2d23220e42cec19c8bcedeb96d
Previous hash: d21ec302d0aa322a933947b2020419e054627dbb0bf074d56deb42b862abedcb
nonce: 270
data: This is the second blockchain.
Proof of work validation: true
Timestamp: 1711788686
hash: 84b859767f63568e9779ce3bee36d93b28a734b4611bca087b30040fdf158140
Previous hash: 6f3c44e96d073df10d86ec1f2c609795cffe2d23220e42cec19c8bcedeb96d
nonce: 463
data: We added Pow in it
Proof of work validation: true
Timestamp: 1711788687
hash: 5ac063f26d47122350219560d39c8fff1f0187206244c5e75e1f7ddc0f4d9104
Previous hash: 84b859767f63568e9779ce3bee36d93b28a734b4611bca087b30040fdf158140
```

```
nonce: 2248
data: We learn Hash in a new way,more deeply.
Proof of work validation: true
Timestamp: 1711788688
hash: 33b8604d40d2ac8f097fcb4ed8a76ef72bd37ae2959cec623ba880d1d6489cd5
Previous hash: 5ac063f26d47122350219560d39c8fff1f0187206244c5e75e1f7ddc0f4d9104
nonce: 246
data: Waiting forward to the next Line!
Proof of work validation: true
```